

Documentation

Table of contents:

Section	Page
1. Purpose of Document	
1.1 Scope	3
1.2 Target Audience	3
1.3 User Manual	3
2. Scientific Context and Software Motivation	
2.1 Directed Evolution Workflow	3
2.2 Computational Challenges	4
3. Design Philosophy	
3.1 Scope of the Web Portal	6
3.2 Overview of System Principles	6
4. Software Architecture	
4.1 Architectural Overview	7
4.2 Architecture Diagram	8
4.3 Data Flow Pipeline and Technologies Used	8
5. Database Design and Schema	
5.1 Overview	9
5.2 Entity-Relationship Diagram	11
5.3 Table Definitions	12
5.4 Key Relationships	15
5.5 Design Decisions	15
6. Core Functional Modules	
6.1 Staging Pipeline	16
6.2 Parsing and Quality Control Module	23

6.3 Analysis Module	25
6.4 Visualisation Module	32
7. Limitations	34
8. Future Development	36
9. References	37

1. Purpose of this document

1.1 Scope

This document describes the internal workings of the Directed Evolution Monitoring Web Portal. The architectural design, database structure, data processing pipeline, and reporting framework are explained in detail in this document to justify the technical decisions we have made during development of the web portal.

The full source code for the web portal is publicly available at:

<https://github.com/sumaiyazainab/group-project-monetary-.git>

1.2 Target Audience

The Directed Evolution Monitoring Web Portal is designed for experimental, synthetic, and computational biologists, as well as protein engineers, who require end-to-end analysis and reliable yet efficient interpretation of multi-generational directed evolution data, specifically from a droplet-based microfluidics protein engineering pipeline. The portal is intended to be easily accessible and traceable, allowing researchers to transform raw experimental data into interpretable biological decisions in one setting.

1.3 User Manual

This documentation focuses on the internal structure, technical details and decisions made when developing the web portal. User-targeted instructions on how to access and make use of the web portal are written separately in the User Manual, hosted at:

<https://sumaiyazainab.github.io/group-project-monetary-/>

The User Manual source files are also available in the docs-site/ directory of the GitHub repository.

2. Scientific context and software motivation

2.1 Directed evolution workflow

Directed evolution (DE) is a laboratory method that mimics years of natural evolution and applies intentional selective pressure to engineer improved molecules such as proteins (e.g. enzymes, antibodies) and nucleic acids.

By performing random mutagenesis to generate library of variants, researchers are able to find rare evolved sequences with superior properties such as improved stability, stronger ligand-target binding or higher catalytic activity.

The process of DE experiment involves the repetition over the three steps listed below, with each cycle progressively aiming to improve the molecule compared to the generation before:

1. Variant Generation:
 - a. Genetic diversity is introduced into the target gene by randomly (chemical-induced, error-prone PCR) or semi-rationally (focusing mutagenesis on specific residues), generating libraries of hundreds up to thousands of variants.
2. Variant Translation:
 - a. Each variant is translated into a protein or RNA molecule, either inside living cells (such as *E. coli* or yeast), where the cell's own machinery produces the molecule, or in a test tube (in vitro transcription-translation) where extracted purified machinery components produce the molecule.
3. Variant Selection:
 - a. Each variant is tested for the desired property using assays that produce a measurable signal. The top-performing variants are selected and rescued, and used as the starting point for the next round of generation and more mutagenesis.

More recently, DE experiments have been applied to different fields of research, expanding from medicine and therapeutics (e.g. improve antibodies for immunotherapy or enzymes for drug production) to industrial biotech (e.g. optimise engineered enzymes to withstand harsh industrial conditions), or even bioremediation (e.g. making enzymes to break down environmental pollutants).

This web portal is specifically designed to support droplet-based microfluidic DE experiments, where droplet-based microfluidic systems can individually encapsulate millions of variants inside water-in-oil droplets, and screen for DNA polymerase activity by measuring DNA yield relative to protein abundance (Manteca et al., 2021).

2.2 Computational Challenges

The following sections outline the challenges raised by droplet-based microfluidic DE experiments that the web-portal addresses:

Challenge 1: Data Scaling and Automation

Improvements in traditional DE techniques, such as the use of droplet-based microfluidic systems, enable the analysis of larger, more complex libraries, resulting in large volumes of data that need to be computationally analysed (Manteca et al., 2021).

With science gradually shifting towards the Design-Build-Test-Learn cycle (Opgenorth et al., 2019), the idea of automated processes has rapidly emerged to overcome limitations from manual analysis workflows, which are incompatible with the large scale of data from modern DE experiments (Packer & Liu, 2015). There is a higher need for automated analysis methods that can feed these large experimental data, from high-throughput methods, directly into computational analysis pipelines, to efficiently make data-driven decisions, and rapidly identify the top variants for further validation.

Our web-portal enables an automated validation staging pipeline, variant mapping and annotation, activity score computation and ranking, and generation of visualisations to assess the distribution of activity scoring results.

Challenge 2: ORF Localisation in Circular Plasmids

In DE experiments, depending on the sequencing method used, circular plasmids can be sequenced at random points, and random linearisation could occur, especially from those arising from short-read sequencing (Illumina) or standard nanopore sequencing. This results in the plasmid DNA sequences beginning at arbitrary nucleotide positions, where the target gene may split and span across the linearised origin.

Our web-portal doesn't assume a fixed linearisation point, strand, or canonical start codon. By treating the plasmid sequence as circular, investigating the reverse strand if not identified in the forward strand, and using a user-selected start codon, the ORF identification takes all of these possibilities into account when identifying exactly where the target gene ORF may sit in the plasmid sequence during experimental staging. The same process is implemented during ORF identification of experimental data variants, before protein alignment and variant annotation are performed.

Challenge 3: Standardised Activity Scoring Metric

In DE experiments, raw measurements, such as fluorescence or activity measurements, cannot be directly compared because they are influenced by technical variability unrelated to true activity. Factors such as batch-to-batch differences, instrument signal drift, and reagent variability can introduce variance that makes cross-generational comparisons unreliable. Additional variables, including differences in cell growth rate, plasmid copy number, and protein expression levels, can also introduce shifts.

For example, when optimising DNA polymerases, a variant can appear highly active because it produces more protein rather than due to the enzyme itself being more efficient.

By designing our web portal to optimise DNA polymerases from droplet-based microfluidics DE experiments, we address this by calculating activity as the ratio of DNA yield to protein yield for each variant, providing a measure of activity. This value is then normalised against the mean activity of wild-type controls within the same generation. This two-step approach reduces potential experimental noise and batch effects, producing an activity score that more accurately reflects intrinsic variant molecule performance for fair comparison and ranking of variants across generations.

Challenge 4: Interpretable Biological Insight

DE experiments produce large longitudinal datasets that contain both variant-level and population-level information. Without appropriate visualisation tools, it is difficult to identify evolutionary trends.

The portal, therefore, transforms raw experimental data into interpretable outputs using three visualisations:

- a Top-10 variant ranking table
- a per-generation activity distribution plot
- a 3D PCA-based activity landscape

These visualisations allow researchers to assess evolutionary progress and identify best-performing variants for further experimentation.

3. Design Philosophy

3.1 Scope of the Web Portal

The portal is specifically designed for droplet-based microfluidic DE pipelines targeting DNA polymerase optimisation, where the functional performance is quantified through DNA yield relative to protein abundance.

The software is also designed to accommodate the biological reality of DE experiments, ensuring biological correctness throughout computational analysis, while generating results that are readily interpretable by researchers without having to leave the web portal.

3.2 Overview of System Principles

Validation Staging Pipeline

The portal follows a validation-first workflow where experimental data cannot be uploaded and analysed until the experimental staging step is successful.

Users must first provide two inputs:

1. A wildtype plasmid FASTA sequence
2. A UniProt accession ID for the target protein

The staging pipeline confirms that the provided UniProt protein sequence is encoded by the gene within the uploaded plasmid before analysis can proceed. This prevents incorrect reference sequences from affecting downstream mutation analysis.

During this stage, the UniProt-retrieved protein feature annotations are also stored and displayed in the results interface. The protein's annotated features, specifically key domains, are useful to identify areas where mutations are most likely to alter important functions. Additionally, this allows for every experiment analysis to be able to trace back to already validated biological data.

Biologically-Informed Sequence Handling

Rather than assuming a canonical ATG start codon, the web-portal accommodates for the two most common alternative start codons (GTG, TTG) regularly found in eukaryotic and prokaryotic expression systems (such as *E.coli*).

This is an important step to eliminate sequence mishandling during downstream alignments especially for variant annotation, or open reading frame (ORF) identification, because ORF nucleotide sequences may begin with GTG/TTG as alternative start codons, but the standard RNA codon table will translate these codons to Valine (V) or Leucine (L) respectively, instead of Methionine (M) as shown in reference protein sequences.

Additionally, rather than assuming the target gene is localised in the middle of the linearised plasmid DNA sequence, the web-portal accounts for the circularity of the plasmid to ensure correct ORF identification during both the experimental staging and the variant annotation processes.

Reproducibility and comparability

A standardised activity scoring metric, normalised to a per-generation baseline, is applied consistently across all experimental data to correct for potential technical variabilities. This ensures that activity scores are directly comparable across generations, and that results are reproducible and auditable. Additionally, all experimental data and results are stored within the user's account, allowing researchers to reference and compare across past experiments.

End-to-end interpretability

From entering raw data → sequence parsing → normalised activity scoring → ranking best performing variants → visualisations (generational activity score distributions and 3D PCA-based landscape), the web-portal is designed so that a researcher does not have to leave the environment, with quick and easy interpretable results generated to efficiently make biological decisions. This is important especially for biologists who may not have the particular bioinformatics expertise, since the portal is able to perform the underlying complex computational analysis and present results in a form that is directly informative for the next experimental decision.

The following section describes the software architecture that implements these principles.

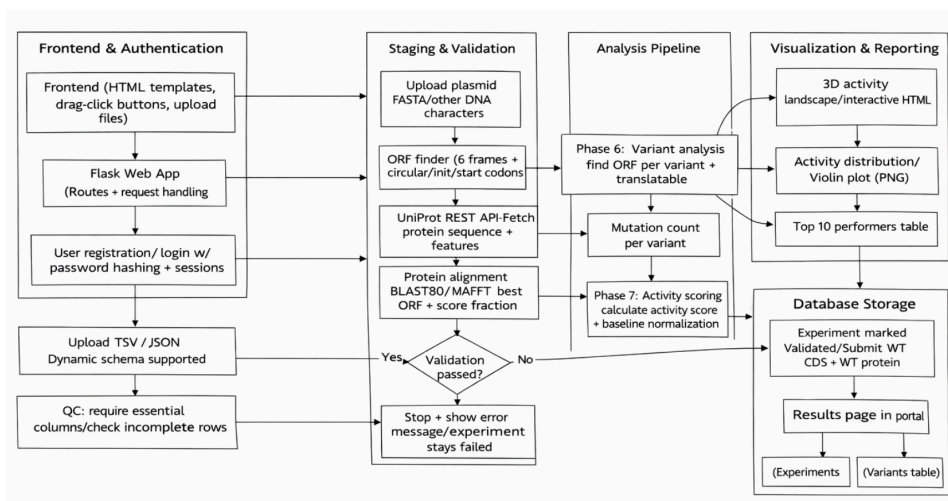
4. Software Architecture

4.1 Architectural overview

The web portal is built as a pipeline, where each stage must be completed before moving to the next step:

Staging → Validation → Parsing/Quality Control (QC) → Analysis → Visualisation/Reporting → Download/Storage

Experimental analysis begins once the reference protein has been successfully validated within the uploaded plasmid sequence. Following that, experimental datasets are parsed, analysed, stored in the database, and visualised within the web interface.



4.2 Architecture diagram

This diagram shows how the system connects the browser interface, Flask backend, external UniProt API, analysis pipeline, and the SQLite database. Users submit inputs through the UI. Flask coordinates the workflow, stores experiment state in the database, triggers analysis when ready, and finally generates visual outputs.

4.3 Data Flow Pipeline and Technologies Used

User inputs are submitted through the web portal interface and received by Flask route handlers. During staging, the Flask backend coordinates UniProt API calls using the Requests library, plasmid FASTA parsing and ORF identification using the Biopython library, and validated results are stored in the SQLite database through SQLAlchemy. Once staging is successful, the experimental data (TSV or JSON file) uploaded by the user is parsed and quality-checked using Pandas. The analysis pipeline then performs ORF detection, translation, and variant classification using Biopython and computes activity scores. Finally, the visualisation module uses Matplotlib to generate per-generation activity score distribution violin plots, and Plotly to generate an interactive 3D PCA-based map from the analysed variant data, rendered in the browser interface.

Below is a summary table of all core technologies used to develop the software. To see how they were used, refer to section **6. Core Functional Modules** for more specific implementations:

Component	Technology	Justification
Web framework	Flask	Lightweight, integrates with SQLAlchemy, provides request handling, routing and template rendering.
ORM	SQLAlchemy	Provides clean object-relational mapping that allows database entities to be defined and queried as Python objects.

Database	SQLite	Serverless, and does not require a separate database process. Appropriate for the expected demands from the web portal.
External API	UniProt REST API	Database source for curated protein sequences and other protein-associated data. Used for experimental staging to retrieve reference protein sequence and feature annotations using a user-provided accession ID.
HTTP Requests	Requests	HTTP Get calls to the UniProt REST API during experimental staging, retrieving JSON response, basic error handling for invalid user-input accession IDs.
Sequence analysis	Biopython	Well-established, peer-reviewed standard library for parsing, manipulating and aligning sequences. This is used throughout the staging and variant annotation pipeline.
Data handling	Pandas	Selected for its compatibility with TSV and JSON input files, efficient data processing, QC (e.g. missing value detection), and DataFrame handling.
Numerical computation	NumPy	Used for array-based operations in the analysis and visualisation pipelines.
Visualisation	Plotly + Matplotlib	Plotly was selected for the interactive 3D plot, and Matplotlib was used for the per-generation activity score distribution plots.
Dimensionality reduction	Scikit-learn PCA	Well-documented standard machine learning library that provides a reliable PCA implementation for the 3D PCA-based map.
K-mer vectorisation	Scikit-learn CountVectorizer	Used to convert protein sequences into k-mer frequency vectors for PCA.

5. Database Design & Schema

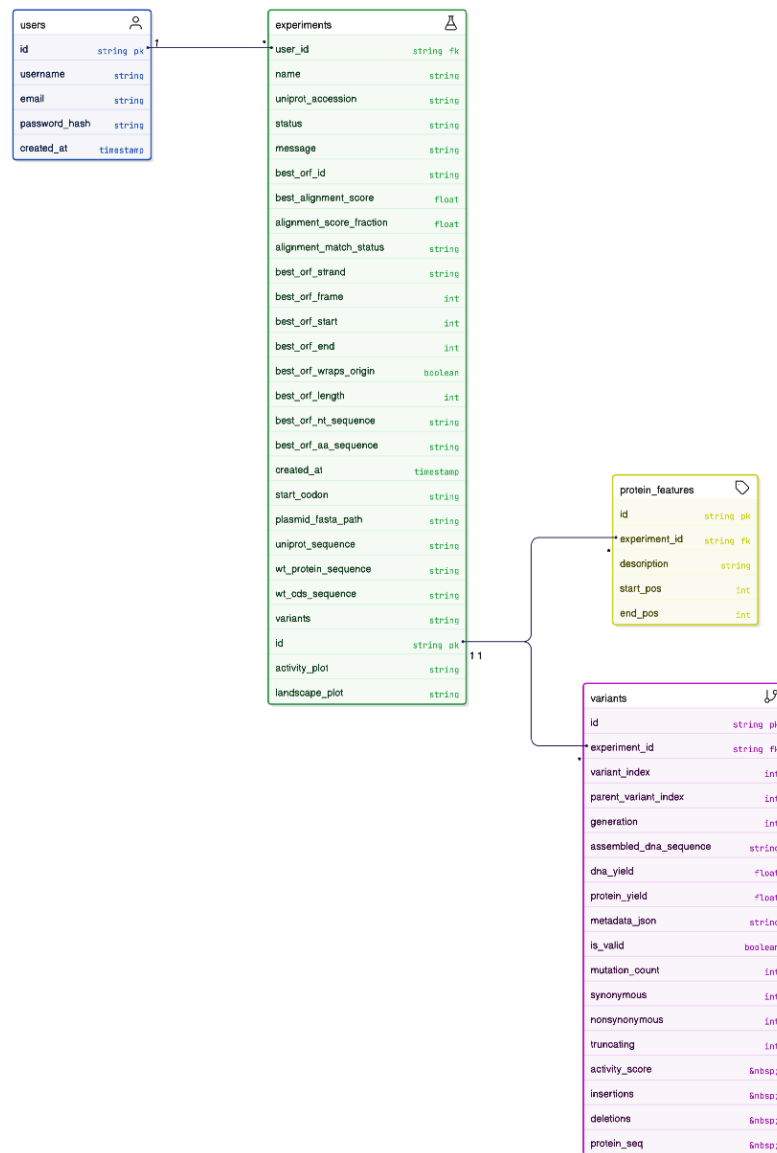
5.1 Overview - what database system is used and why

The database system used for the portal is SQLite relational database, managed through SQLAlchemy ORM within the Flask application. This setup simplifies the development and helps maintain consistency across the codebase. SQLite was chosen as it is serverless and does not require a separate database server process.

The schema is organised around four core entities: User, Experiment, Protein features and Variant. A user owns one or more experiments, and each experiment stores all associated

staging data such as the validated reference protein, best ORF candidate detected within the user-uploaded plasmid FASTA, and staging status directly within the experiment record. The Protein features table stores UniProt-retrieved functional domain annotations, and is linked to the parent Experiment table. The sequenced experimental variants are also linked to the parent Experiment table and annotated with variant classifications and activity score results. Functional domain annotations are also retrieved from UniProt during staging and stored within Experiment table.

5.2 Entity-Relationship Diagram



5.3 Table Definitions

- User

Stores registered user accounts. Email and username are enforced as unique to prevent duplicated registrations. Passwords are stored as hashes. The created_at field records when the account was created.

Field	Description
id	Primary key
username	Unique username
email	Unique email address
password_hash	Hashed password
created_at	Account creation timestamp

- Experiment

This is the central entity of the schema, representing one DE analysis run submitted by a user. Each experiment tracks a processing status (staged, parsing, analysed, or error) and an associated message to reveal potential errors or warnings to the user on the web-portal. The best_orf_id and best_alignment_score fields record the candidate ORF with the highest alignment score against the protein reference, and are only filled after ORF identification during the staging pipeline. Protein reference sequence data (e.g. UniProt protein sequence, wildtype protein sequence, and wildtype CDS) are stored directly in the Experiment table. The activity_plot and landscape_plot field store the generated visualisation outputs linked to the experiment.

Field	Description
id	Primary key
user_id	Foreign key linking to the User table
name	Experiment name entered by the user
uniprot_accession	UniProt accession ID provided by the user
status	Processing status: pending, match, no_match, uploaded, analysed, or error
message	Error or warning message returned to the user
best_orf_id	ID of the best-matching ORF identified during staging
best_alignment_score	Alignment score fraction of the best-matching ORF

created_at	Experiment creation timestamp
start_codon	Start codon selected by the user
plasmid_fasta_path	File path to the uploaded wildtype plasmid FASTA
uniprot_sequence	Reference protein amino acid sequence retrieved from UniProt
wt_protein_sequence	Translated wildtype protein sequence from the best-matched ORF
wt_cds_sequence	Wildtype coding DNA sequence extracted from the plasmid
variants	Relationship linking to associated Variant records
activity_plot	Stored per-generation activity distribution plot output
landscape_plot	Stored 3D PCA-based activity landscape plot output

- Protein features

Stores UniProt-fetched annotated functional domains in the reference protein, linked to its parent experiment. During staging, all annotated feature annotations are extracted from the UniProt record alongside the reference protein sequence. The extracted features are then filtered by domain type, and stored as individual records. Each protein feature record contains a description of the domain function, and amino acid start and end position of the domain.

These annotations displayed are useful for researchers to cross-reference variant mutations identified from the top 10 variants table, against known functional regions of the protein. An example of this would be, a nonsynonymous mutation, compared to synonymous, within an important functional domain is more likely to influence catalytic activity.

Field	Description
id	Primary key
experiment_id	Foreign key linking to the parent Experiment table
description	Description of the annotated functional domain retrieved from UniProt (e.g. 5'-3' exonuclease)
start_pos	Amino acid position where the annotated domain begins in the reference protein sequence
end_pos	Amino acid position where the annotated domain ends in the reference protein sequence

- Variant

Stores each sequenced experimental variant, linked to its parent experiment. Each variant is annotated with its generation number, DNA and protein yield measurements, mutation

classification results, and computed activity score. The `is_valid` flag records if a variant has passed quality control. The `metadata_json` field stored additional metadata columns from the uploaded experimental data file outside of the seven required fields (Mentioned in **Section 6.2 Parsing & QC Module**). The `is_wt_control` flag identifies wildtype control reads, which are used as the per-generation normalisation baseline.

Field	Description
<code>id</code>	Primary key
<code>experiment_id</code>	Foreign key linking to the Experiment table
<code>variant_index</code>	Unique identifier for the variant (Plasmid_Variant_Index)
<code>parent_variant_index</code>	Identifies the parent variant
<code>generation</code>	Directed evolution generation number
<code>assembled_dna_sequence</code>	Full assembled DNA sequence of the variant
<code>dna_yield</code>	DNA quantification value (fg) - specific for DNA polymerase optimisation DE experiments
<code>protein_yield</code>	Protein quantification value (pg) - specific for DNA polymerase optimisation DE experiments
<code>metadata_json</code>	Additional metadata columns stored as JSON
<code>is_valid</code>	Boolean flag indicating whether the variant passed QC
<code>mutation_count</code>	Total number of mutations relative to wildtype
<code>synonymous</code>	Count of synonymous substitutions - substitution alters nucleotide sequence but does not change the resulting amino acid in the protein
<code>nonsynonymous</code>	Count of nonsynonymous substitutions - substitution alters nucleotide sequence and changes the resulting amino acid in the protein
<code>truncating</code>	Boolean flag for premature stop codon detection
<code>activity_score</code>	Normalised Activity Score relative to per-generation wildtype mean
<code>insertions</code>	Count of insertion events
<code>deletions</code>	Count of deletions events
<code>protein_seq</code>	Translated variant protein sequence

5.4 Key Relationships

- A User owns many Experiments, but each Experiment belongs to exactly one User
- Each Experiment can have many Protein Features, as a reference protein can have multiple annotated functional domains retrieved from UniProt
- Each Experiment can have many Variants spanning multiple generations of DE data
- Alongside experimental variants, wildtype control variants are also stored in the Variant table, identified by the Control field in the uploaded data

5.5 Design Decisions

5.5.1 Experiment as the Central Entity

The database is structured so that Experiment is the central entity, with all reference sequence data stored directly on the experiment record rather than in separate tables. The UniProt protein sequence, plasmid FASTA-detected wild-type protein sequence and CDS are stored as fields on the Experiment table. This ensures all data from a single analysis run is directly traceable to one experiment record and simplifies the potential deletion of an experiment cleanly.

5.5.2 Reference Protein Feature Records Stored Separately from Experiment

All annotated features of a reference protein fetched from the UniProt record during staging, is filtered to retain only annotations that are domain types. These filtered functional domain annotations are stored in a separate Protein Feature table linked to the Experiment by a foreign key, instead of directly inside the Experiment table. This decision keeps the parsing logic generic, and allows for future implementations of filtering for other types of annotation types such as binding sites or active sites. Additionally, the annotations are kept in a separate table, because an experiment and a single reference protein can give rise to multiple domain annotations. As a separate record, each domain can be stored, retrieved, and displayed individually without encoding multiple values inside a single database field.

All other reference sequence data, including UniProt protein sequence, best ORF sequence, and metadata, are stored directly inside Experiment table because they produce singular data per experiment, and are frequently used during downstream variant annotation and activity scoring pipelines.

5.5.3 User-Exclusive Data Access

All experiments are linked to the user's unique registered ID through a foreign key relationship. This enforces data privacy at the schema level where users can only view, modify, or delete experimental data that they have initialised and uploaded by themselves.

5.5.4 Storing All Variant Experimental Data

All experimental variants are stored in the Variant table rather than only keeping top-performing variants. This is important and essential for generating population-level

visualisations, specifically the per-generation activity score distribution plot and the 3D PCA-based fitness landscape, and ensures full reproducibility from the stored dataset.

5.5.5 Storing Mutation Results Directly on Variant Records

Mutation classification results, synonymous, nonsynonymous, insertion, deletion, and truncating counts, are stored as separate fields directly in each Variant record rather than in a separate mutation table. This avoids additional joins during ranking and visualisation queries, and keeps all variant-level data retrievable in a single query.

5.5.6 Metadata JSON Field

The Metadata JSON field stores all non-required column (e.g. batch labels, other experimental details specific to their lab or setup) from the user-provided experimental data file. This keeps the schema stable and able to accommodate different experimental formats regardless of what extra information different users include in their files. Not only does this keep a faithful record of the original experimental data, this also ensures that data isn't partially lost and unrecoverable for user transparency, and if the portal adds new features in the future, a column not currently needed right now may be useful in the future.

5.5.7 Storing Visualisation Outputs at Experiment Level

The generated two visualisation outputs, the activity distribution plot and 3D PCA-based landscape, are stored directly in the Experiment table (activity_plot and landscape_plot) at the point they are generated, to ensure the portal only ever computes them once. Every time the user wants to view their results, the visualisations can be easily retrieved and rendered in the portal.

6. Core Functional Modules

6.1 Staging Pipelines

6.1.1 Purpose

The staging module takes a user's input (UniProt accession ID and plasmid FASTA) to validate a new experiment by verifying that the selected target gene is encoded in the uploaded plasmid FASTA sequence. This module occurs before experimental data analysis to ensure biological correctness for downstream variant analysis and activity scoring.

The objectives are:

1. Retrieve the UniProt protein using the user-provided UniProt accession ID, and extract the relevant information:
 - Amino acid sequence
 - Location and function of key features, specifically domains
2. Validate and parse the user-uploaded plasmid FASTA sequence.
3. Verify the retrieved protein reference is encoded by the plasmid FASTA sequence:

- I. Identify candidate open reading frames (ORFs) within the plasmid and translate into an amino acid sequence.
 - II. Find the best candidate ORF by aligning each amino acid sequence to the reference protein sequence, determined by the highest alignment score
 - III. To validate, determine if the best candidate ORF is a match (`match_found`) or not a match (`no_match_found`) to the reference protein sequence by setting an alignment score fraction cut-off threshold (0.85)
6. Store the validated best ORF sequence, match status, and corresponding metadata for downstream analysis.

6.1.2 Design Rationale

The staging process was designed to be:

- Generalizable: Supports any UniProt accession ID.
- Robust to circular plasmids: ORFs spanning the origin are correctly detected.
- Robust to alternative bacterial start codons: Canonical start codon ATG, and alternative bacterial start codons (GTG, TTG) are supported depending on user input.
- Computationally efficient: Length pre-filtering reduces sequence alignment cost and noise during sequence validation.
- Biologically meaningful: Protein alignment uses BLOSUM62 scoring.
- Dynamic global alignment threshold: The system identifies coding sequences by similarity to the UniProt-fetched reference protein sequence, and allows for leniency in cases of unexpected variants in the wildtype plasmid FASTA by using a score fraction threshold (0.85).

6.1.3 Workflow Overview

The staging process follows this pipeline:

1. UniProt Protein Record Retrieval
 - Protein amino acid sequence
 - Location and function of key domains
2. Plasmid FASTA Upload and Validation
3. Candidate ORF Identification
4. Length-based Pre-filtering
5. Best ORF Selection by Global Protein Alignment
6. Data storage

Each step is described below.

6.1.4 UniProt Reference Protein Record Retrieval

The system queries the UniProt REST API using a user-provided accession ID and extracts the associated reference protein information. The API request is executed using the Python **requests** library, which performs a HTTP GET request to the UniProt endpoint and retrieves the protein record in JSON format.

From the returned record, the information extracted are:

- Amino acid protein sequence and length of sequence
- Annotated protein features, with downstream filtering applied to DOMAIN-type annotations

The UniProt JSON response is parsed into a Python dictionary and processed using the **pandas** library. All feature annotations are flattened into a single pandas DataFrame containing the feature type, description, and positional coordinates (start and end nucleotides).

Feature filtering is applied afterwards by selecting rows where the feature type equals DOMAIN. This design keeps the parsing logic generic, and allows the same stored feature dataset to support future implications when filtering for other annotation types (e.g., motifs, regions, or binding sites).

This step verifies the reference wildtype protein used for downstream validation processes.

If the user-provided accession ID is invalid or the protein sequence cannot be extracted from the UniProt record, the system raises either a `ValueError` or `RuntimeError`, and the staging pipeline terminates.

Technologies Used:

- Python: To implement API communication, error handling, data parsing.
- UniProt REST API: To retrieve protein records associated with the user-provided accession ID, returning a structured protein information as a JSON format.
- Requests library: To perform HTTP GET requests for retrieving protein information.
- Pandas library: To transform UniProt feature annotation into a structured DataFrame for efficient filtering and sorting for downstream analysis.
- JSON data format: The format in which UniProt fetched protein records is retrieved.

6.1.5 Plasmid FASTA Upload and Validation

The user-uploaded plasmid sequence is provided as a FASTA file and parsed using the biopython SeqIO module, before the candidate ORF identification step. This validation step reads the FASTA record, extracts the plasmid header and nucleotide sequence, and ensures the sequence is composed only of valid uppercase letters.

The validation process performs the following checks:

- The FASTA file must contain exactly one sequence.
- The nucleotide sequence must only contain valid DNA characters (A, T, C, G, optionally N for unknown bases).

After successful validation, the extracted information is stored in a structured dictionary and stored in the database under the Plasmid entity:

- FASTA header identifier
- plasmid nucleotide sequence
- plasmid sequence length (base pairs)

If the FASTA file contains more than one sequence, no sequences, or invalid nucleotide characters, the system raises a **ValueError**, and the staging pipeline terminates before entering the ORF detection stage.

Technologies Used:

- Python: Implementation language for input validation and sequence processing
- Biopython (SeqIO): FASTA parsing and sequence extraction
- FASTA format: Standard sequence file format

6.1.6 Candidate ORF Identification

After successful FASTA parsing and validation, the plasmid nucleotide sequence undergoes ORF identification to detect potential protein-coding regions that may correspond to the reference protein retrieved from UniProt.

Because plasmids are **circular DNA molecules**, ORF detection must account for coding sequences that may span the plasmid origin. To enable this, the validated nucleotide sequence is concatenated to itself to produce a linear representation of the circular plasmid, allowing ORFs that span across the origin to also be detected:

```
circular_seq = sequence + sequence
```

Candidate ORFs are then identified across all six reading frames:

- Forward frames: f1, f2, f3
- Reverse complement frames: r1, r2, r3

The reverse complement strand is generated using nucleotide complement mapping and sequence reversal operations implemented in Python.

ORFs identification criteria

1. User-selected start codon

Translation initiation is defined using a user-selected start codon through a check-box interface (support start codons: ATG, GTG, TTG), to accommodate a wider range of expression systems beyond those with the canonical ATG start codon.

2. DNA translation

Candidate nucleotide sequences are translated into amino acid sequences using **Biopython's Seq.translate()**, which applies the **generic codon translation table**.

A custom Python translation function is specifically implemented because the generic codon translation table used by Biopython's translate module is unable to accommodate alternative start codons (GTG, TTG), which will translate these to their normal amino acids according to the generic codon translation table. In actuality, when an ORF begins with any valid start codon, the start codon is always forced to start with the Methionine (M) amino acid, and UniProt protein sequences always begin with M. Therefore, this custom function ensures consistent sequence comparability during staging.

Example:

GTG (normally V) translated as M

TTG (normally L) translated as M

Unknown or invalid codons, as a result of ambiguous nucleotides (N), are translated to X, so the pipeline remains robust instead of failing silently.

3. Stop codons

Candidate ORFs stop as soon as one of these stop codons is reached, and the terminal stop codons (*) are excluded from translation to match the sequence format used by UniProt protein sequences: TAA, TAG, TGA

4. Minimum ORF length: 50-amino acids

During candidate ORF filtering, a 50-amino acid minimum length threshold is applied to reduce the number of short, non-functional, and random ORFs that were generated across the six reading-frames.

This improves specificity and computational efficiency during protein alignment (the next step) by reducing false positives.

ORF metadata extraction for storing:

Each ORF that satisfies the above requirements have the following records (metadata) stored as dictionary objects within a structured list for easy downstream comparison:

Key	Value/Description
strand	Coding strand: + / -
frame	f1 / f2 / f3 / r1 / r2 / r3
start_nt	Nucleotide position of ORF start in the plasmid sequence
end_nt	Nucleotide position of ORF end in the plasmid sequence
wraps_origin	Boolean flag identifying ORFs that spans the plasmid origin
length_nt	Length of ORF (nucleotide)
orf_nt	DNA coding sequence
length_aa	Length of ORF (amino acid)
orf_aa	Translated protein sequence, excluding stop codon

Technologies Used:

- Python: Implementation of ORF detection algorithm and sequence processing
- Biopython (SeqIO, Seq): FASTA parsing, sequence manipulation, translation

- FASTA format: Input format for plasmid DNA sequences
- Custom six-frame ORF scanning algorithm: Detection of ORFs, which are protein-coding regions, within the circular plasmid DNA

6.1.7 Length-based Pre-filtering

Before performing computationally expensive protein alignments, candidate ORFs are first pre-filtered by length compatibility against the UniProt reference protein sequence length.

For each candidate ORF, the translated amino acid length (`length_aa`) is compared to the UniProt reference protein length (`len(uniprot_protein_seq)`). An ORF is only retained if the candidate ORF sequence length is within $\pm 20\%$ of the UniProt sequence length.

- Minimum length: $0.8 \times \text{UniProt protein length}$
- Maximum length: $1.2 \times \text{UniProt protein length}$

This step reduces the alignment search space, improves runtime performance, and removes random truncated or excessively long candidates to the expected protein length.

If no ORFs pass the filter, the pipeline falls back to aligning and scoring all ORFs in the next step.

Technologies Used:

- Python: Implementation of length threshold computation and candidate filtering logic
- UniProt reference protein: Baseline length is used to define the compatibility tolerance window

6.1.8 Best ORF Selection by Global Protein Alignment

After length-based pre-filtering, the best candidate ORF is computed by finding the candidate ORF sequence with the best alignment score against the UniProt reference protein sequence.

Protein similarity scoring is implemented using **Biopython's Bio.Align.PairwiseAligner**, and computed using the following configurations:

- Global alignment (`aligner.mode = "global"`)
- BLOSUM62 substitution matrix (`substitution_matrices.load("BLOSUM62")`)
- Affine gap penalties (`gap open: -10, gap extend: -0.5`)

Global alignment was selected because the expected ORF should represent the full-length protein, and local alignment would allow partial matches, which would increase false positives.

Score fraction:

A score fraction is obtained when a candidate ORF is aligned to the UniProt reference sequence:

1. Candidate ORF raw alignment scores against the UniProt reference sequence are computed:
 - `score_orf = aligner.score(uniprot_seq, orf_seq)`
2. A maximum theoretical score is computed by aligning the UniProt reference sequence to itself:
 - `score_max = aligner.score(uniprot_seq, uniprot_seq)`
3. A score fraction is finally computed by normalising the ORF alignment score to the maximum theoretical score
 - `score_fraction = score_orf / score_max`
4. Compare with the previous candidate sequence's score fraction, if the score fraction improves (increases), keep the current candidate ORF. If the previous candidate's score fraction is better, keep the previous candidate to compare to the next candidate ORF.

These steps are repeated until the ORF with the highest score fraction is found within the candidate ORF list, or once a score fraction of 1 (100% alignment with reference protein) is found

ORF Acceptance Threshold for Plasmid Validation:

The best matching ORF is accepted as a good match if **score_fraction $\geq$ 0.85**. This threshold is designed to:

- Balance sensitivity and specificity when identifying the correct ORF
- Allow moderate sequence divergence (mutations, indels)
- Reject unrelated ORFs with weak similarity
- Reduce false positives from large random ORFs aligning by chance

If the best matching ORF does not meet this threshold, the experiment status is set to `no_match`, and the user receives an explanatory message that there were no candidate ORFs aligned sufficiently well to the provided UniProt reference protein.

Technologies Used:

- Python: For filtering, scoring and best ORF selection
- Biopython `Bio.Align.PairwiseAligner`: Pairwise protein alignment and scoring
- Biopython `Bio.Align.substitution_matrices`: Loading and applying BLOSUM62
- BLOSUM62: Standard amino acid substitution matrix for protein similarity scoring
- Affine gap penalty model: Applies a stricter penalty to gap openings than gap extensions, to reflect typical biological insertion and deletion patterns

6.1.9 Data Storage

Upon successful ORF match and validation from the user-uploaded plasmid sequence to the user-provided UniProt accession ID reference protein, the following information is stored in the system database to ensure reproducibility and full traceability of each staging decision:

Table	Stored Data
experiment	Experiment metadata, best_orf_id and best_alignment_score, and staging status
protein_reference	UniProt accession ID, reference protein sequence
protein_feature	Annotated UniProt protein features
plasmid	Plasmid FASTA header, sequence, sequence length
orf_candidate	All detected ORF candidates

6.1.10 Error Handling and Status States

During the execution of the staging pipeline, the state of the experiment is tracked (experiment.status field), and the experiment may transition into different staging pipeline outcomes:

Status	Description
error	invalid UniProt or FASTA
no_orfs_found	No valid ORFs detected
no_match_found	ORFs detected, but none sufficiently similar
staged	successful ORF identification

6.2 Parsing & QC Module

6.2.1 Purpose

The Experimental Data Parsing & QC module operates after experiment staging and before the Analysis & Visualisation pipeline. It ensures that only validated experimental data is inserted into the SQL database.

6.2.2 File Parsing Implementation

The system supports TSV and JSON formats using the Pandas library. Pandas was selected for its robust handling of missing values, efficient DataFrame structures, and compatibility with SQL insertion workflows.

6.2.3 Design Rationale

The parser validates only essential analytical fields while allowing additional metadata columns. This ensures flexibility and future compatibility with evolving experimental designs. Certain fields are required for correct analysis.

Essential fields include:

- Plasmid_Variant_Index
- Parent_Plasmid_Variant
- Directed_Evolution_Generation
- Assembled_DNA_Sequence
- DNA_Quantification_fg
- Protein_Quantification_pg
- Control

6.2.4 Workflow Overview

Two levels of validation are performed:

- Column-level validation (essential columns must exist).
- Row-level validation (rows with missing essential values are removed).

Quality Control is executed before database insertion to preserve scientific integrity and prevent corrupted records from influencing mutation classification or Activity Score calculations.

6.2.5 Database Design and Integrity

Validated records are inserted into an SQL relational database (SQLite). Foreign key relationships link experimental data to both the user and experiment tables.

Pre-insertion validation ensures transactional consistency and protects downstream analytical accuracy.

6.2.6 Contribution to Overall System

This module forms the foundation of the analytical workflow. By ensuring clean and validated data, this module enables reliable mutation detection, accurate Activity Score computation, and meaningful visualisation outputs.

6.3 Analysis Module

6.3.1 Purpose

The Analysis Module operates after the Parsing & QC module and before the Visualisation pipeline. It is responsible for variant ORF localisation, DNA-to-protein translation, mutation classification, and activity score computation for each sequenced variant relative to the validated wildtype reference established during staging.

The objectives are:

1. Locate the target gene ORF within each variant plasmid sequence using the validated wildtype ORF as a reference anchor
2. Translate variant DNA sequences into protein sequences, accommodating alternative start codons
3. Align each variant protein to the wildtype reference protein to restore positional correspondence between residues
4. Classify mutations as synonymous, nonsynonymous, insertion, deletion, or truncating
5. Compute a normalised Activity Score for each variant that reflects intrinsic catalytic efficiency, independent of expression level or batch variability
6. Store all annotated results for downstream visualisation and ranking

6.3.2 Design Rationale

The analysis pipeline was designed to be:

- **Robust to sequence divergence:** ORF localisation uses an approximate seed-matching algorithm rather than exact matching, so the target gene can still be identified when variants carry multiple mutations
- **Biologically accurate in mutation classification:** Alignment is performed at the protein level and projected back to the codon level, ensuring that insertions and deletions do not cause reading frame misinterpretation during synonymous/nonsynonymous classification. The downstream logic assumes that gaps indicate indels and that mismatches indicate substitutions if scoring were wrong, mutations could be misclassified, codons misaligned, or fake truncations detected
- **Specific to DNA polymerase engineering:** The Activity Score is formulated as a DNA yield per unit protein metric. This is biologically meaningful specifically for polymerase variants where the catalytic output is DNA synthesis, and would not be appropriate for other enzyme classes with different catalytic outputs
- **Robust to batch variability:** Activity scores are normalised to per-generation wildtype control means, correcting for assay drift and inter-generation variability

6.3.3 Workflow Overview

The analysis pipeline follows this sequence:

1. ORF Localisation in Variant Plasmid Sequences
2. DNA-to-Protein Translation
3. Global Protein Alignment
4. Codon-Level Alignment Construction
5. Mutation Classification and Annotation
6. Activity Score Computation
7. Data Storage

Each step is described below.

6.3.4 ORF Localisation in Variant Plasmid Sequences

The portal resolves variant ORF localisation by aligning the validated wildtype CDS directly to the variant plasmid sequence, providing an unambiguous, strand-aware, origin-spanning ORF anchor before any variant annotation is performed.

Because exact matching fails when mutations exist in the early portion of the gene, the module uses an approximate seed-matching approach. The `best_approx_match_pos()` function aims to find where a short seed sequence best matches inside a longer plasmid sequence, even if there are some mismatches. It slides a window the same length as the seed along the sequence, counts the number of mismatches at each position, and keeps track of the position with the fewest mismatches. Mutated genes will not perfectly match the reference gene, so instead of exact matching, this allows substitutions, mild divergence, and robust ORF detection.

Once the best matching start position is identified, `extract_cds_to_first_stop()` extracts a CDS from the circular plasmid starting at that position and stopping at the first in-frame stop codon. This treats the plasmid as circular so indexing wraps around the end of the string, ensuring the ORF is biologically valid. These two steps are coordinated by `find_orf_by_seed_and_stop()`, which uses the first `seed_len` bases of the reference ORF as a search probe, calls `best_approx_match_pos()` to find the best match, and then calls `extract_cds_to_first_stop()` to extract the full ORF. Without this, any mutation in the first part of the gene could break ORF detection entirely.

If the seed cannot be placed with sufficient confidence, a warning is logged, and that variant is skipped, preventing the pipeline from crashing on problematic inputs.

Technologies Used:

- Python: To implement the seed-matching algorithm and circular plasmid indexing
- Biopython (SeqIO, Seq): Sequence manipulation and codon extraction
- Logging: Timestamped audit trail recording of which variants were processed, skipped, or failed

6.3.5 DNA-to-Protein Translation

Candidate nucleotide sequences are translated into amino acid sequences using a custom translation function `translate_with_alt_start_with_stop()`. Standard Biopython translation is not used directly because the generic codon translation table is unable to accommodate alternative start codons (GTG, TTG), which would be translated to their normal amino acids (Valine and Leucine respectively). In actuality, when an ORF begins with any valid start codon, that codon is always forced to Methionine (M), and UniProt protein sequences always begin with M. Therefore this custom function ensures consistent sequence comparability during variant annotation.

The function translates codons in triplets using the standard genetic code, forces the first codon to Methionine regardless of which valid start codon is present, and preserves internal stop codons as characters rather than truncating at the first stop enabling downstream truncation detection. Unknown or invalid codons arising from ambiguous nucleotides (N) are translated to X so the pipeline remains robust rather than failing silently.

Technologies Used:

- Python: Custom codon-by-codon translation logic
- Biopython (Seq): Codon lookup and genetic code translation

6.3.6 Global Protein Alignment

After translation, each variant protein is globally aligned against the validated wildtype protein using Biopython's `Bio.Align.PairwiseAligner` via the `global_align_proteins()` function. Global alignment was selected because the expected ORF should represent the full-length protein local alignment would allow partial matches and increase false positives. A global alignment aligns the entire length of both proteins from start to end, forcing the algorithm to account for all amino acids and all indels, not just a local matching region.

The alignment is configured with the scoring scheme below:

Parameter	Value	Rationale
Alignment mode	Global	Full-length end-to-end alignment required
Match score	+2	Rewards conservation, pushed identical residues to align
Mismatch score	-1	Lightly penalises substitutions, reflecting plausible mutations
Gap open penalty	-5	Strongly discourages new gaps unless necessary
Gap extend penalty	-1	One long indel is more likely than many small ones

The total alignment score is computed by summing each of these contributions, and the higher-scoring alignment is chosen. This scoring scheme directly affects mutation counts, synonymous vs nonsynonymous classification, and truncation detection. The function returns two gapped aligned strings, one for the wild type and one for the variant, which are used in the subsequent codon alignment step.

Technologies Used:

- Biopython (Bio.Align.PairwiseAligner): Pairwise global protein alignment and scoring

6.3.7 Codon-Level Alignment Construction

Since mutation classification happens at the codon level and not just the protein level, `build_codon_aligned_cds()` converts the protein alignment back into a codon-aligned DNA alignment. If there is a gap in protein space, a triple gap --- is inserted in DNA space. The function walks through the aligned protein strings, keeping two pointers one for wildtype DNA codons and one for variant DNA codons. For each aligned position: if the protein equals an amino acid, it appends the next codon; if the protein equals -, it appends ---. This ensures codons stay in-frame, gaps are biologically meaningful, and synonymous vs nonsynonymous mutations can be detected. Without this step, codon 50, for example, in the wild type would not correspond to codon 50 in the variant after an indel event.

6.3.8 Mutation Classification and Annotation

Using the codon-aligned sequences, `classify_mutations_codon_aligned()` counts and classifies mutations by iterating through codon pairs and applying the following logic:

- Codons are identical: No mutation
- Codons differ, amino acids identical: Synonymous substitution
- Codons differ, amino acids differ: nonsynonymous substitution
- WT codon is - - - : Insertion
- Variant codon is - - - : Deletion
- Variant codon translates to : Truncating (premature stop)

Human-readable mutation labels (e.g. T599S, A600del) are generated by `extract_mutation_annotations()`, which walks through the aligned protein strings position by position, tracks the true wildtype position ignoring gaps, and produces biologically interpretable mutation notation.

The full pipeline is coordinated by `variant_analysis()`, which loads and QC-checks the input data, identifies the wildtype control row, extracts and translates the wildtype ORF, then loops over all non-control variants performing ORF localisation, translation, alignment, codon projection, and classification for each. If ORF extraction fails for a variant, a warning is logged, empty results are output, and the pipeline continues to the next variant preventing the pipeline from crashing. Each variant yields: `protein_seq`, `mutations`, `aa_positions`, `mutation_count`, `synonymous`, `nonsynonymous`, `insertions`, `deletions`, and `truncating`.

Technologies Used:

- Python: For mutation classification logic and annotation label generation
- Biopython (Seq): Codon translation for synonymous/nonsynonymous determination
- Pandas: Result merging and export to CSV and JSON
- Logging: Audit trail for skipped or failed variants

6.3.9 Activity Score Computation

Biological Justification

The experiment measures DNA output (how much plasmid DNA is produced) and protein output (how much polymerase enzyme is present). The Activity Score is formulated specifically for DNA polymerase-directed evolution, where the catalytic output of the enzyme is DNA synthesis. DNA yield therefore, directly reflects polymerase activity, a more efficient polymerase produces more DNA per unit enzyme, making it an appropriate proxy for catalytic efficiency. This metric would not be appropriate for other enzyme classes where catalytic output is not DNA.

However, raw DNA yield alone is scientifically unreliable because it confounds two independent variables:

DNA yield = $f(\text{enzyme efficiency, enzyme concentration})$

A variant that is poorly efficient but highly expressed may appear to outperform one that is highly efficient but poorly expressed. Statistically, protein concentration is a nuisance variable and catalytic efficiency is the latent variable of interest. The Activity Score removes this confounder by normalising DNA yield by protein abundance.

A good Activity Score must satisfy the following criteria:

1. **Biological interpretability** - it must correspond to something biologically meaningful, enabling comparisons such as "Variant X is 3× more efficient than the wildtype"
2. **Normalisation across experiments and generations** - absolute measurements depend on reagent concentrations, instrument settings, batch effects, and temperature, so the score must be relative rather than absolute
3. **Penalise trivial solutions** - the metric must penalise variants that only succeed through overexpression and reward variants that produce more DNA with less protein, to avoid selecting expression artefacts instead of better enzymes
4. **Robustness to noise** - experimental readouts are noisy due to droplet variation and fluorescence noise, so the metric averages WT baselines and avoids division by very small numbers
5. **Comparability across generations** - generation 1 and generation 8 scores must be comparable, requiring a consistent baseline and consistent scaling

Formula

The Activity Score is computed in two steps:

Step 1 - Raw activity per variant:

$$E_i = D_i / P_i$$

D_i = DNA yield variant i

P_i = protein yield of variant i

This estimates the DNA produced per unit protein but it suffers from scale differences between experiments, unknown units and batch effects.

Step 2 - Define the WT efficiency:

$$1- E_{WT} = \langle D_{WT} / P_{WT} \rangle$$

$$2- \text{Activity Score}_i = (D_i / P_i) / E_{WT}$$

Interpretation:

Score	Interpretation
≈ 1.0	Equivalent activity to wildtype
> 1.0	Improved activity relative to wildtype
< 1.0	Reduced activity relative to wildtype

Per-Generation vs Global Baseline

The pipeline computes both a per-generation ($E_{WT,g}$) baseline and a global baseline (E_{WT} (= mean of all WT controls)).

The per-generation baseline is used for variant ranking each generation has its own experimental context, its own parent construct, and its own assay conditions, and the wildtype control is re-measured each generation specifically to capture drift. Per-generation normalisation is beneficial for selection efficiency per round, mutational step effect and local fitness landscapes. These considerations are more faithful to directed evolution theory, where each variant is compared to its immediate evolutionary parent.

The global baseline, the mean of all wildtype control rows across all generations, treats the wild type as a distribution of baseline measurements across the experiment rather than a single construct. Using only a single generation-0 baseline would ignore assay drift and accumulating batch effects. This provides a lower-variance estimator of wildtype activity and is used for cross-generational trend plotting, where a consistent fixed reference is required.

Implementation

The activity scoring pipeline is implemented across four coordinated functions:

- `identify_wt_rows(df)` - selects rows where `Control == True`, encoding the experimentally defined wildtype baseline
- `compute_raw_activity(df)` - computes $E_i = D_i / P_i$ for all rows, adding a `raw_activity` column

- `compute_global_wt_baseline(df)` - computes single WT reference using all WT rows. It calls the function above, takes their raw activity and computes the mean.
- `compute_generation_wt_baselines(df)` - computes separately for each generation, producing a generation-specific baseline map
- `compute_activity_score(df, baseline_mode)` - computes the final normalised Activity Score using either 'per-generation' or 'global baseline', adding an `activity_score` column

These are coordinated by the `activity_scoring()` pipeline, which applies all steps in sequence and returns a fully scored DataFrame ready for variant ranking and visualisation.

Technologies Used:

- Python: Activity score computation and baseline strategy logic
- Pandas: DataFrame grouping by generation, column assignment, and export
- NumPy: Array-based mean computation for baseline calculations

6.3.10 Data Storage

Upon completion of variant analysis and activity scoring, results are merged with the original dataset and stored per variant:

Field	Description
<code>protein_seq</code>	Translated variant protein sequence
<code>mutations</code>	Human-readable mutation labels (e.g. T599S)
<code>aa_positions</code>	Affected amino acid positions
<code>mutation_count</code>	Total number of mutations relative to wildtype
<code>synonymous</code>	Count of synonymous substitutions
<code>nonsynonymous</code>	Count of nonsynonymous substitutions
<code>insertions</code>	Count of insertion events
<code>deletions</code>	Count of deletion events
<code>truncating</code>	Boolean flag for premature stop codon
<code>raw_activity</code>	Uncorrected DNA/protein ratio
<code>activity_score</code>	Normalised Activity Score relative to per-generation wildtype mean

6.4 Visualisation Module

The Visualisation Module converts processed experimental data into interpretable outputs that allow users to evaluate evolutionary progress, identify high-performing variants, and explore relationships between sequence diversity and functional activity. After staging, data parsing, and activity score calculation are completed, the portal generates three primary visual outputs:

- Top 10 Performing Variants Table
- Activity Score Distribution per Generation (Violin Plot)
- 3D Activity Landscape (Sequence Diversity vs Activity Surface)

Each visualisation highlights a different aspect of the experiment: individual variant performance, population-level trends, and the broader sequence–function landscape.

6.4.1 Top 10 Performing Variants Table

The Top 10 Performing Variants table provides a ranked summary of the highest-performing DNA polymerase variants identified during the DE experiment. DE experiments typically generate hundreds or thousands of variants across multiple generations, making it difficult to quickly identify the most promising candidates. By ranking DNA polymerase variants according to their wild-type-normalised Activity Score, the table highlights variants that demonstrate improved catalytic efficiency relative to the wild-type enzyme.

Each entry includes the variant identifier, parent variant, generation number, DNA and protein quantification values, the calculated Activity Score, and mutation statistics relative to the wild-type sequence. Mutation counts and classifications are generated using Biopython, where variant DNA sequences are translated into protein sequences and aligned to the validated wild-type reference sequence. This allows mutations to be identified and categorised as synonymous, nonsynonymous, or truncating. The total mutation count represents the number of amino acid changes present in the variant, while the mutation classification provides additional insight into whether these sequence changes alter the protein structure.

The Activity Score reflects the enzymatic efficiency of DNA polymerase variants by normalising DNA output by protein expression levels. DNA polymerase variants with higher scores, therefore, produce more DNA per unit of protein, suggesting improved catalytic performance rather than simply increased expression. By combining functional measurements with mutation information, the table enables researchers to assess how sequence variation influences enzyme performance.

6.4.2 Activity Score Distribution per Generation

The Activity Score Distribution per Generation visualisation provides a population-level view of functional performance across successive rounds of DE. While the Top 10 table highlights

individual high-performing variants, this plot captures how the entire population of variants changes over time under selective pressure. This is important because DE operates as a population-based optimisation process, where improvements emerge gradually across generations rather than from a single variant alone.

A violin plot is used to visualise the distribution of Activity Scores for each generation. This type of plot was chosen because it simultaneously displays the density distribution of values and key summary statistics, providing more information than simple bar charts or box plots. Each violin represents all non-control variants within a specific generation. The width of the violin at any vertical position reflects the density of variants with that Activity Score, while markers indicate the mean and median values for the generation.

This visualisation allows researchers to observe how the distribution of enzymatic activity evolves. For example, an upward shift in the median Activity Score suggests that the overall population is improving, indicating that selection pressure is successfully enriching for higher-performing variants. A narrowing distribution may indicate convergence toward an optimal solution, while a wider distribution suggests continued exploration of sequence space. In some cases, a bimodal distribution may appear, indicating the coexistence of both high- and low-performing subpopulations.

If successive generations show progressively higher median Activity Scores, this indicates that the selection process is successfully enriching for variants with improved catalytic performance.

The plot is implemented using Python, NumPy, and Matplotlib. Variant data is first filtered to exclude wild-type control measurements so that the distribution of engineered variants is not compressed by baseline values. Variants are then grouped by their `Directed_Evolution_Generation`, and the corresponding Activity Scores are extracted into arrays using NumPy. These arrays are passed to `matplotlib.pyplot.violinplot()`, with the parameters `showmeans=True` and `showmedians=True` enabled to display central tendency alongside the distribution shape.

The resulting plot displays generation number on the x-axis and the WT-normalised Activity Score on the y-axis, providing an intuitive visual summary of how functional performance evolves across the experiment.

6.4.3 3D PCA-based Activity Landscape

The 3D Activity Landscape provides a global view of how sequence diversity relates to functional performance across all variants in a DE experiment. Unlike the Top 10 table (which highlights individual best variants) and the violin plot (which summarises generation-wide trends), the landscape combines every variant into a single sequence–function map. In this plot, each variant is positioned in a 2D “sequence space” (PCA Dim 1 and PCA Dim 2), and its height (Z-axis) represents the WT-normalised Activity Score. The smoothed surface acts like a fitness landscape: peaks indicate regions of higher activity (potential solutions reached by selection), while valleys represent lower-performing sequence regions. This helps assess whether high-activity variants cluster (suggesting convergence on a shared solution), whether multiple peaks exist (indicating alternative

solutions), or whether the landscape forms a broad plateau (suggesting mutational robustness).

To build the landscape, each variant's DNA sequence is first translated into a protein sequence using Biopython, so that diversity is measured at the functional (amino-acid) level. Protein sequences are then converted into numerical feature vectors using 3-mer (k-mer) counting with CountVectorizer, which captures local composition without requiring alignment. These high-dimensional vectors are reduced to two dimensions using PCA, producing the X and Y coordinates used for plotting. A K-nearest neighbour (KNN) regressor is then trained to predict Activity Score from PCA coordinates, and predictions across a grid are used to generate a smooth surface (go.Surface). The measured variants are overlaid as points (go.Scatter3d) with hover text showing variant ID, generation, Activity Score, and mutation count. The output is interactive and is saved as an HTML file for embedding in the portal. This visualisation is useful because it reveals patterns that are not visible from ranking or distribution plots alone, such as local optima, clusters of related high-performers, and whether evolution is exploring widely or converging strongly. It remains an approximation: PCA compresses sequence differences into two components, and KNN smoothing depends on data density, so sparse regions of sequence space may be less reliable.

7. Limitations

Although the Directed Evolution Monitoring Web Portal provides a functional end-to-end pipeline for analysing DE experiments, several limitations remain in the current implementation.

1. Activity Score Metric Specificity

The Activity Score metric is currently specific to DNA polymerase-DE experiments, where DNA yield is a biologically meaningful proxy for catalytic activity. The metric assumes that DNA abundance reflects DNA polymerase enzymatic activity, and that normalising by protein abundance isolates catalytic efficiency from expression-level differences. Although valid for polymerase systems, the same activity score metric would not be appropriate for other classes of enzymes and molecules, which would require a different output measurement to reflect catalytic activity. The portal is therefore not currently generalisable to directed evolution experiments targeting other enzyme types, and future development would be required to support configurable biological measure inputs and alternative activity scoring pipelines.

2. Staging pipeline match status threshold

The current staging pipeline has several limitations. The protein alignment threshold is set at $\text{score_fraction} \geq 0.85$, and may accommodate most plasmid constructs, but may not be suitable for highly divergent plasmid constructs.

3. Translation: assuming standard genetic code

Although a custom translation function is being used to accommodate alternative start codons, the current pipeline assumes the standard genetic code and does not support

alternative genetic codes from other biological systems, for example, by specific organisms or organelles.

4. Computational Performance

The current implementation performs sequence processing, alignment, and mutation analysis using Python in-memory operations. While sufficient for moderate datasets, analysis time increases as the number of variants and generations grows. The system has not yet been optimised for very large libraries or high-throughput industrial datasets.

5. Simplified Activity Score Model

The unified Activity Score is calculated as DNA yield/protein yield, normalised using the per-generation control baseline's mean DNA yield/protein yield. While this provides a biologically meaningful estimate of relative catalytic efficiency, the current model remains simplified.

The current implementation:

- Assumes a linear relationship between DNA output and enzymatic activity
- Does not incorporate regression-based modelling approaches
- Does not account for potential non-linear biological effects
- Does not include statistical confidence intervals or uncertainty estimates

As a result, the Activity Score is best interpreted as a comparative ranking metric rather than a detailed mechanistic model of enzyme performance.

6. Limited Handling of Experimental Replicates

The current system does not explicitly support the analysis of replicate measurements beyond the WT control baseline. As a result:

- Replicate datasets cannot be automatically aggregated
 - Variance, standard deviation, or error estimates cannot be calculated
- Statistical comparisons between generations cannot be performed

This limits the statistical robustness of the conclusions that can be drawn from the dataset.

7. Mutation Impact Interpretation

Mutations are currently classified at the sequence-level (e.g., synonymous, nonsynonymous, truncating). However, a deeper functional interpretation is not yet incorporated.

The current implementation:

- Does not analyse the structural effects of mutations
- Does not predict impacts on protein stability
- Does not integrate functional domain context into mutation interpretation
- Does not incorporate whole-population level evolutionary interpretation (e.g. mutational hotspots, conserved regions)

Therefore, biological interpretation remains sequence-based rather than evolutionary or structure-informed. Future improvements could incorporate more in-depth variant amino acid labelling, protein structural data, and evolutionary-level visualisations to identify mutational signatures.

8. Future development

Several extensions could further improve the analytical robustness and scalability of the Directed Evolution Monitoring Web Portal.

1. Configurable Activity Metric

The current Activity Score pipeline hardcodes DNA yield/protein abundance as the enzyme activity ratio, limiting the portal to only DNA polymerase DE applications. Future development would be to allow users to specify which experimental measurement columns map to the expected catalytic output when starting a new experiment. This would extend the web portal's applicability to a wider range of enzyme classes, whilst keeping other parts of the web portal (such as staging and visualisation) the same.

2. Dynamic alignment threshold

The current alignment acceptance threshold (`score_fraction` ≥ 0.85) is a fixed value. Potential enhancements would be to implement an adaptive thresholding strategy, where the threshold changes depending on specific alignment characteristics such as sequence length and substitution similarity score distribution.

This approach would improve the robustness of the staging pipeline, especially involving highly mutable plasmids, engineered or evolutionarily divergent proteins.

3. Support for Alternative Genetic Codes

Some biological systems, such as specific bacteria, archeal and mitochondrial genomes, use alternative genetic codes instead of the standard genetic code. Future enhancements of the portal's staging pipeline would allow users to select an alternative codon table when initialising an experiment, and use Biopython's translation table framework (`Bio.Data.CodonTable`) to allow alternative codon tables to improve the applicability of the web-portal to a wider range of organisms and expression systems.

4. Improved Activity Score Modelling

The current Activity Score provides a simple ranking metric but could be enhanced with more advanced statistical approaches. Future versions may include:

- Linear and non-linear regression models
- Log-transformed activity scaling
- Confidence interval estimation
- Outlier detection

These improvements would provide more robust and biologically informative activity measurements.

5. Replicate-Aware Analysis

The current system does not fully support experimental replicates beyond WT controls. Future development could include:

- Aggregating replicate measurements for each variant
- Calculating mean activity values and variance
- Displaying standard deviation or error bars
- Performing statistical tests (e.g., ANOVA across generations)

This would strengthen the statistical reliability of experimental conclusions.

6. High-Performance Sequence Processing

To support larger datasets, computational performance could be improved through:

- Vectorised NumPy operations
- Parallel processing (multiprocessing)
- GPU-accelerated sequence alignment for large datasets

These improvements would enable efficient analysis of large variant libraries.

7. Machine Learning–Driven Variant Ranking

Future versions may incorporate machine learning approaches to complement the rule-based Activity Score, such as:

- Predictive models based on mutation patterns
 - Detection of epistatic interactions
- Prediction of high-performing variants before experimental validation.

9. References:

Manteca, A., Gadea, A., Van Assche, D., Cossard, P., Gillard-Bocquet, M., Beneyton, T., Innis, C.A. & Baret, J.-C. (2021) Directed Evolution in Drops: Molecular Aspects and Applications. *ACS Synthetic Biology*. 10 (11), 2772–2783. doi:[10.1021/acssynbio.1c00313](https://doi.org/10.1021/acssynbio.1c00313).

Opgenorth, P., Costello, Z., Okada, T., Goyal, G., Chen, Y., Gin, J., Benites, V., de Raad, M., Northen, T., Deng, K., Deutsch, S., Baidoo, E., Petzold, C., Hillson, N., García Martín, H. & Beller, H. (2019) Lessons from Two Design-Build-Test-Learn Cycles of Dodecanol Production in *Escherichia coli* Aided by Machine Learning. *ACS Synthetic Biology*. 8. doi:[10.1021/acssynbio.9b00020](https://doi.org/10.1021/acssynbio.9b00020).

Packer, M.S. & Liu, D.R. (2015) Methods for the directed evolution of proteins. *Nature Reviews Genetics*. 16 (7), 379–394. doi:[10.1038/nrg3927](https://doi.org/10.1038/nrg3927).